

INF01151 – Sistemas Operacionais II

Aula 18 - Algoritmos distribuídos & Sincronização de relógios físicos

Prof. Alberto Egon Schaeffer Filho

Introdução

- Processos em um sistema distribuído cooperam entre si
- Tempo é uma questão importante/interessante em sistemas distribuídos
 - Tempo é a quantidade que desejamos medir de forma precisa
 - Transação de e-Commerce envolve eventos no computador do vendedor e no computador do banco, importante uso de timestamps precisos
 - Algoritmos dependem da sincronização de relógios
 - Manter consistência de dados distribuídos, coordenar acesso a recursos distribuídos, ordenar mensagens e eliminar updates duplicadas
- **Problema**: Não existe relógio físico absoluto!!
 - Necessita métodos para sincronizar **aproximadamente** relógios
 - Usando troca de mensagens
 - Limite na precisão de timestamps de eventos em nodos diferentes
 - Ordem em que cada par de eventos aconteceu (ou se aconteceram simultaneamente)

Sincronização de relógios físicos: o problema

- O tempo não é ambíguo em sistemas centralizados
 - Para obter hora, os processos A e B fazem chamada de sistema
 - Se A executa antes de B, então $t_A < t_B$
 - Existe um relógio absoluto que pode determinar t_A e t_B
 - Porém, problema não é trivial em sistemas distribuídos
 - Cada computador tem o seu relógio
 - Não podem ser sincronizados perfeitamente
 - Sincronização aproximada
 - Relógios atrasam/adiantam

➤ **Aula de HOJE: relógios físicos**

➤ **Tópico da próxima aula: relógios lógicos** (podem ser usados para definir uma ordem de eventos sem medir o tempo físico)

Relógios, eventos e estados

- Ordenação de eventos que ocorrem em um único processo p_i
 - Processo p_i executa em um processador e não compartilha memória
 - Processo p_i tem um estado s_i , que inclui estado de suas variáveis
 - Processos só se comunicam por troca de mensagens
 - Execução de p_i consiste em série de ações:

- Operação `send` em p_i
 - Operação `receive` em p_i
 - Alteração do estado s_i
- } Eventos

ATENÇÃO: ordenamento $\rightarrow_i$ é bem definido mesmo que processo seja multithread, pois assumimos execução em um único processador

- Sequência de eventos em um único processo pode ser colocado em ordem total
 - Bem definida pois processo p_i executa em um único processador
- $e \rightarrow_i e'$: se e somente se e executa antes de e' em p_i

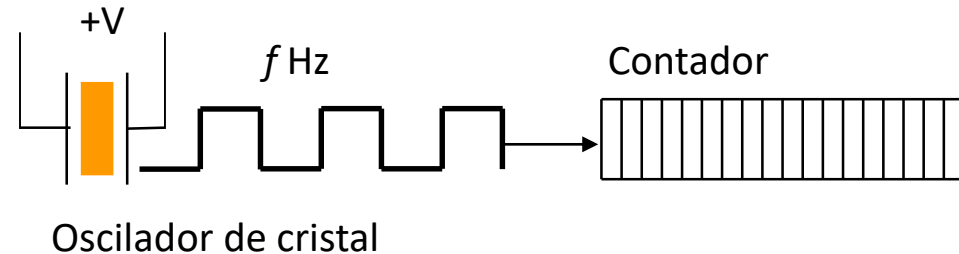
$\text{history}(p_i) = h_i = \langle e_i^0, e_i^1, e_i^2, \dots \rangle$

Relógios físicos

- Mas como atribuir timestamps?

- Cada computador tem seu próprio relógio físico

- Conta o número de oscilações em um cristal a determinada frequência, armazena em registrador



- Sistema operacional lê o valor do relógio físico, $H_i(t)$, escala e adiciona *offset* para produzir o *clock* de software $C_i(t)$

$$C_i(t) = \alpha H_i(t) + \beta$$

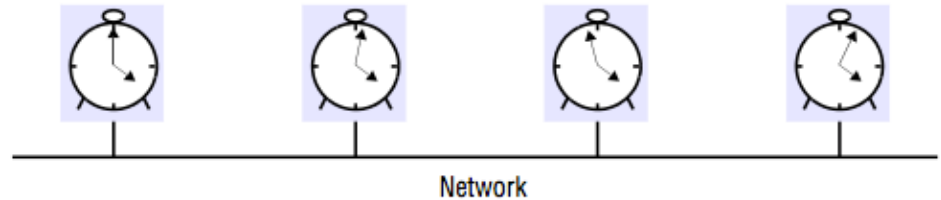
- $C_i(t)$ mede o tempo t real (físico) para processo p_i
 - E.g., número de 64-bits com número de milliseconds desde referência
 - Em geral, $C_i(t)$ será diferente de t por falta de precisão

Problemas: clock skew e clock drift

- Relógios físicos tendem a não concordarem perfeitamente...

- Sujeitos a **clock skew**

- Diferença instantânea entre leituras de dois relógios

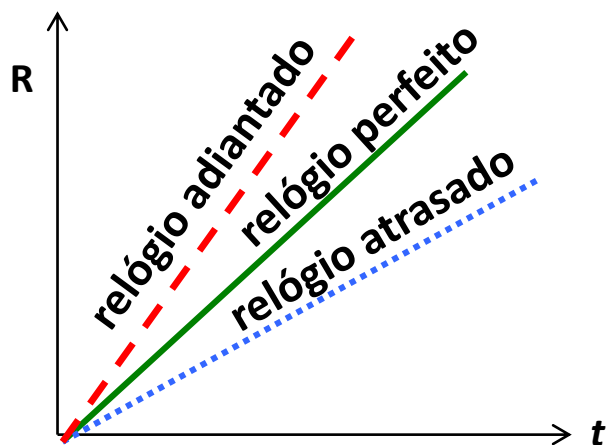


- Relógios baseados em cristal também sujeitos a **clock drift**

- Contam o tempo a taxas diferentes e, portanto, divergem
 - Osciladores estão sujeitos a variações físicas, mudanças de temperatura, etc
 - Possível compensar essa variação (α e β), mas não eliminá-la
 - Necessidade de acertos periódicos
- Relógios comuns baseados em quartz crystal
 - 10^{-6} segundos/segundos, diferença de 1 segundo a cada 1.000.000 segundos (11,6 dias)

Clock skew e clock drift

- Defasagem do relógio em relação a um relógio ideal (perfeito)
 - Relógio adiantado e relógio atrasado



Drift rate é a taxa de deslocamento entre um relógio local e um relógio de referência (ideal)

Diferença instantânea é a distorção (*skew*)

Fatores de correção

$$C_i(t) = \alpha H_i(t) + \beta$$

Relógio de hardware
(*clock ticks*)

Tempo Universal (curiosidades)

- Relógios de computadores podem ser sincronizados com fontes externas de alta precisão
- Relógio **atômico**
 - Relógios físicos mais precisos utilizam osciladores atômicos
 - Precisão de 10^{-13}
 - Standard second: 9.192.631.770 períodos de transição do átomo de Césio 133 (Cs^{133})
- Relógio **astronômico**
 - Base de tempo costumava ser o segundo solar médio
 - Definidos em termos da rotação da terra em torno de si mesmo, e de sua rotação ao redor do sol
 - Período de rotação da terra ao seu redor está se tornando maior
 - Efeito marés, efeitos atmosféricos, correntes de convecção, etc
 - Tempo astronômico e atômico tendem a sair de sincronismo
 - Segundo solar possui um drift em relação ao atômico...

Tempo Universal Coordenado (UTC)

- Coordinated Universal Time (UTC): padrão internacional
 - Baseado no tempo atômico
 - Ocasionalmente, um *leap second* → é inserido p/ mantê-lo sincronizado com tempo astronômico
- Sinais UTC são sincronizados e **broadcast** é feito regularmente
 - **Estações terrestres de rádio**
 - Sinais recebidos tem precisão de **0.1-10 milliseconds**
 - **Satélites (incluindo GPS)**
 - Sinais recebidos tem precisão de **1 microsecond**
- Computadores com receptores podem sincronizar seus relógios com esses sinais



Sincronização de relógios físicos

- Sincronização externa

- Ajustar os relógios C_i em relação a uma fonte externa $S(t)$
- Quando é necessário saber data/hora de eventos em sistema distribuído

$$|S(t) - C_i(t)| < D \text{ para } i = 1, 2, \dots, N$$

Relógios C_i “*são precisos*” dentro de um limite D

- Sincronização interna

- Ajustar os relógios em função dos relógios dos outros
- Objetivo é ter visão “igual” do tempo mesmo que difira do tempo real

$$|C_i(t) - C_j(t)| < D \text{ para } i, j = 1, 2, \dots, N$$

Relógios C_i “*concordam*” dentro de um limite D

Considerações gerais

- Relógios com sincronização interna não estão necessariamente sincronizados externamente
 - Podem sofrer **drift coletivo**, mesmo que concordem uns com os outros
- Porém, se sistema for sincronizado externamente com limite D
 - Também estará sincronizado internamente com um limite $2D$
- Noção de correção:
 - Relógios não dão saltos muito grandes:
 - Taxa de drift ($\rho > 0$) dentro de limite conhecido
 - Relógios não podem voltar no tempo:
 - Monotonicidade: $t' > t \Rightarrow C(t') > C(t)$
- Ajuste em software sem alteração do relógio físico: α e β

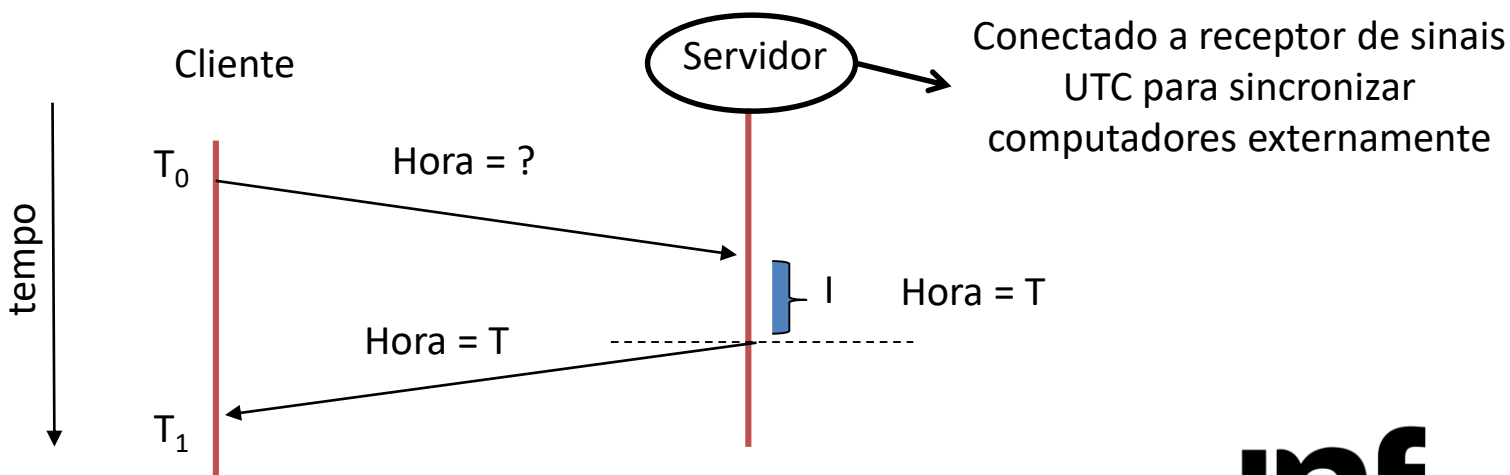
Importante: noção de correção não implica que o relógio deva marcar a hora exata!

Algoritmos de sincronização de relógios

- Modelo de falhas
 - Por colapso: quando pára de “tiquetaquear”
 - Bizantina (arbitrária): quando fornece valores errados
 - Year 2038: signed 32-bit integer representa número de segundos desde 00:00:00 de 1 janeiro 1970... contador irá “estourar” às 03:14:07 UTC de 19 janeiro 2038
 - Quebra da monotonicidade
 - Relógios com baterias fracas (*drift* se torna muito alto)
- Algoritmos de sincronização
 - Servidor passivo (Cristian, 1989)
 - Servidor ativo (Berkeley, 1989)
 - Network time protocol (NTP, 1985)

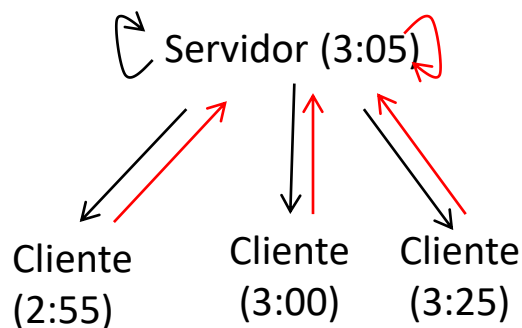
Algoritmo centralizado: servidor passivo (Cristian)

- Algoritmo de Cristian [1989]
 - Utiliza um servidor de “hora” para sincronizar computadores externamente
 - Atende requisições de “hora” fornecendo sua hora local (T)
 - Nova hora no cliente = $T + (T_1 - T_0)/2$
 - onde T_0 é a hora local da requisição e T_1 a hora local do recebimento da resposta

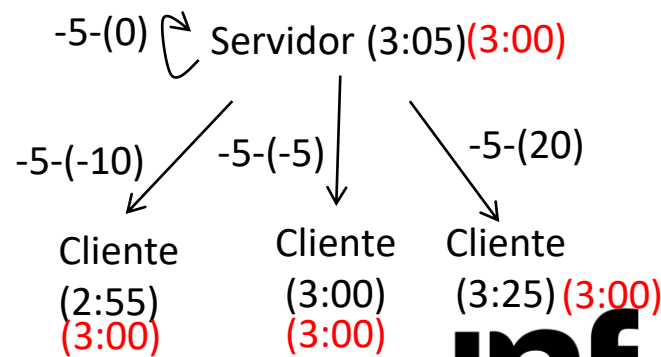


Algoritmo centralizado: servidor ativo (Berkeley)

- Algoritmo Berkeley [1989]
 - Objetivo é sincronização interna de um grupo de máquinas
 - Servidor – *master* – é escolhido por uma eleição (tolerância a falhas)
- Funcionamento
 - Periodicamente o servidor pergunta a hora aos clientes
 - Calcula média dos valores e informa diferença entre a média e os relógios locais
 - Envia a diferença a ser corrigida, não um valor de hora absoluto. Por que?

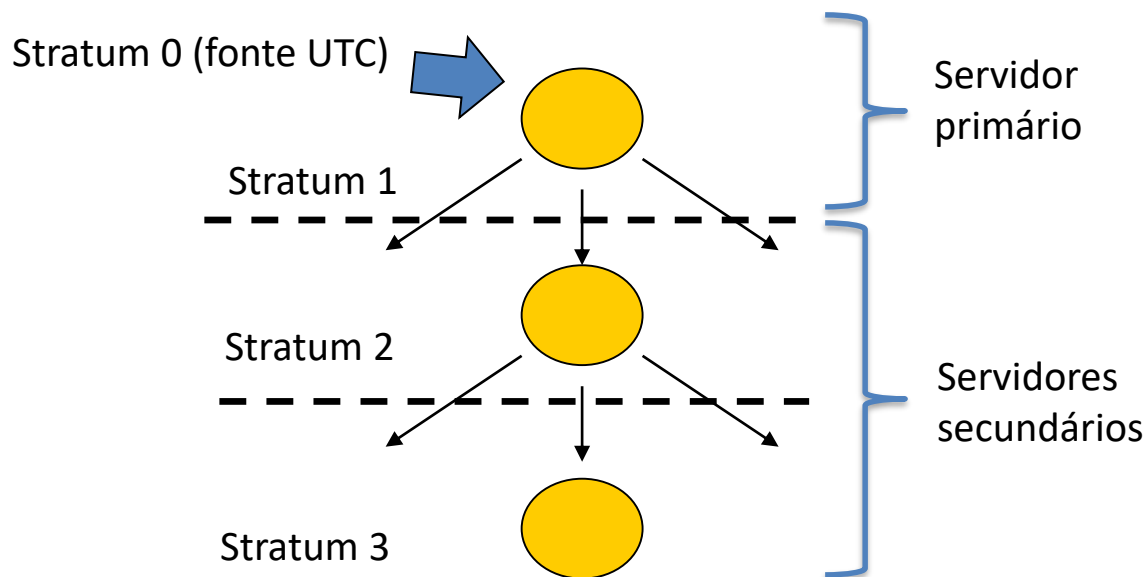


$$\begin{aligned} 3:05 - 3:05 &= 0 \\ 2:55 - 3:05 &= -10 \\ 3:00 - 3:05 &= -5 \\ \del{3:25 - 3:05 = +20} \end{aligned}$$
$$\text{média} = \frac{0 - 10 - 5}{3} = -5$$



Estudo de caso: Network Time Protocol (NTP)

- Projetado para sistemas distribuídos de grande porte (Internet)
 - Baseado em UDP (porta 123) e em operação desde 1985
 - Versão corrente: NTPv4 (RFC 5905)
- Organizado em uma estrutura hierárquica (*strata*)
 - Quanto mais próximo do topo, melhor a precisão



Objetivos do NTP

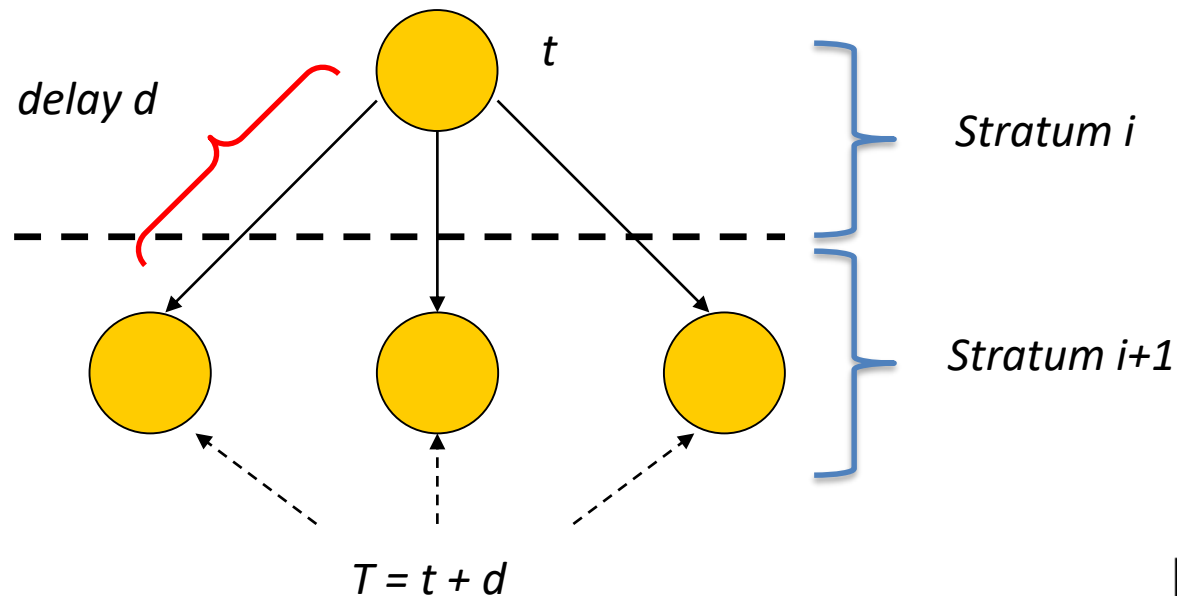
- ① Oferecer serviço para que clientes na Internet possam se sincronizar precisamente com base no UTC
 - Larga-escala, delay grande e variável pode ocorrer
 - Emprega técnicas estatísticas para filtrar atrasos e determina qualidade de dados
- ② Serviço confiável, deve continuar operacional mesmo na presença de falhas
 - Redundância de servidores e de rotas, reconfiguração de servidores
- ③ Permitir re-sincronização frequente de clientes
 - Estrutura hierárquica dos servidores aumenta escalabilidade
- ④ Proteção para evitar mal funcionamento em caso de interferências (acidentais ou propositais)
 - Autenticação e integridade das mensagens NTP

Funcionamento do NTP

- Baseado em três informações
 - **Clock offset**: diferença entre dois relógios (usado para ajuste)
 - **Round Trip delay**: RTT
 - **Dispersão**: máximo de erro permitido entre um relógio local e o relógio do servidor de nível superior
- Disponibiliza 3 modos de sincronização entre servidores NTP
 - Multicast
 - Cliente-servidor
 - Simétrico

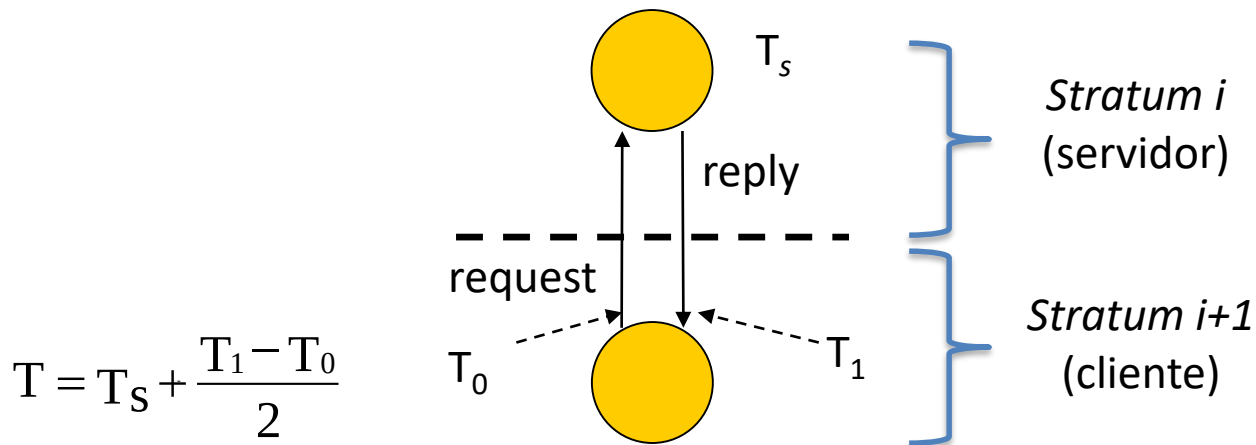
Modo multicast

- Usado em redes locais de alta velocidade
 - Rede precisa oferecer suporte multicast
- Um ou mais sevidores periodicamente fazem multicast de t
 - Receptores ajustam seus relógios assumindo um pequeno *delay*
 - Em geral, baixa precisão, devido à necessidade de se estimar d



Modo cliente-servidor

- Operação similar ao algoritmo de Cristian
 - Um servidor aceita requisição de outros servidores, e as responde com seu clock local
 - Em geral, melhor precisão que no modo multicast
- Alternativa quando não há multicast
 - Também normalmente usado em ambientes de redes locais

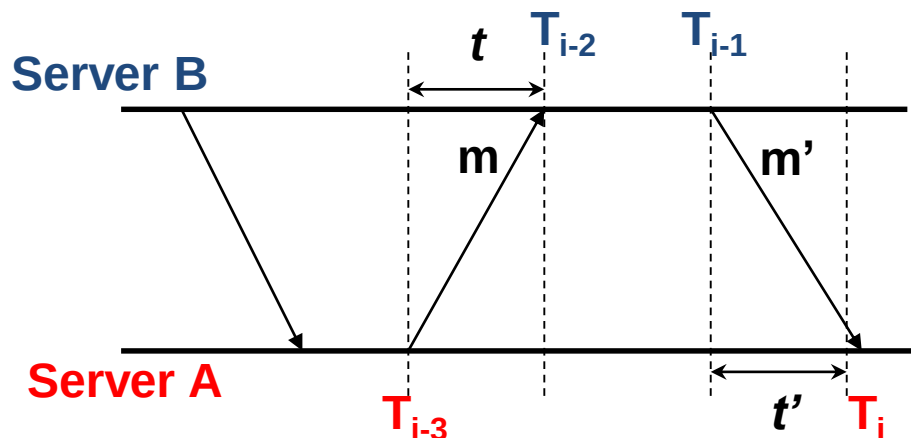


Modo simétrico

Informações adicionais:

- Utiliza filtragem de dados, e seleção de peers
- Precisão da sincronização
 - Na Internet: algumas dezenas de ms
 - Em LANs: 1ms

- Utilizado quando maior precisão é necessária
 - Níveis mais altos na hierarquia (com número de stratum baixo)
 - Associação entre par de servidores
 - Dados de tempo são retidos para melhorar a precisão com o tempo
- Cada mensagem possui timestamps de eventos de msgs recentes:
 - Tempo local do momento da emissão da mensagem anterior
 - Tempo local do momento da recepção da mensagem anterior
 - Tempo local do momento da emissão da mensagem corrente



Para cada par de mensagens m e m' , NTP calcula:

o_i = offset estimado de B em relação a A

d_i = delay no envio das msgs (t e t')

$$T_{i-2} = T_{i-3} + t + o \quad \text{e} \quad T_i = T_{i-1} + t' - o$$

$$d_i = t + t' = T_{i-2} - T_{i-3} + T_i - T_{i-1}$$

$$o = o_i + (t' - t)/2, \quad \text{onde} \quad o_i = (T_{i-2} - T_{i-3} + T_{i-1} - T_i)/2$$

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 14 (seções 14.1, 14.2 e 14.3)